Feature matching for high-resolution neuron-level brain imaging of larval zebrafish

Ira Ceka[1], Drew Robson[2], Jennifer Li[2]

RoLi Lab
Rowland Institute of Science
Harvard University Cambridge, MA

August 2016

Abstract

Learning and memory are two of the most perplexing, complex processes unfolded by the animal brain, and of specific interest are the individual neurons that contribute to these processes. Humans have strived to understand such a level of brain anatomy and function for decades, but only recently did researchers develop the proper technological tools necessary for making neuron-level brain analysis possible. In particular, the development of multilevel microscopy and advanced optogenetic tools has paved the way for in-depth study of the animal brain- specifically, that of the larval zebrafish, which is of particular interest for brain analysis due to its transparent and small size. With these advanced tools come high-resolution brain images, and consequently, the demand for efficient computational imaging analysis. For the duration of modern high-level brain imaging, intensity-based (rather than feature-based) registration was deemed to be the most effective means of analysis.[1] When applied to large volumes of data, however, intensity-based registration proves to be expensive. We strive to tackle a challenge that not many people have a solution for: comparing volumes against volumes inexpensively. To do this, we explore the efficiency of several computational feature detection/matching methods and the significance they hold in analyzing brain images - ultimately pushing us in the direction of understanding behavior at the neuron-level.

1 Introduction

Feature detection methods have - for long - been a topic of interest amongst computer science researchers. Because image analysis permeates many sub disciplines, efficient computational techniques continue to be seen as necessary.

Indeed, this proves to be true for research that involves the generation of large data sets. For neuron-level brain imaging, however, feature-based registration was seen as ineffective due to the nature of the images. Because brain images are not characterized by distinct features, intensity-based registration is often used for processing - even against processing of large volumes. As a method, however, intensity-based registration is highly expensive against large volumes, and because time and resources are valuable, it is worth it to explore the effectiveness of feature-based methods. Well known methods such as SIFT and SURF were commonly used and deemed ineffective for registration purposes, but new methods such as

BRISK, FREAK, AGAST, FAST, KAZE, and AKAZE, could yield promising results. In our quest to find and apply the best computational feature matching method, we study such characteristics as performance, match number, and accuracy. Then, we apply the winning method in the 3-D perspective, with transformed images, and eventually - most importantly - to images with 6-degrees of freedom.

The stack with which the feature-based methods were tested represents a fixed, 2-D version of a larval fish-brain. The size of the stack was 239 images, each image being 1800 x 1796 pixels. In this set up, the images are aligned so that the first few images correspond to the top of the brain, frame 120 (halfway point) corresponds to the very bottom of the brain, and from the halfway-point onward, to the top of the brain again. This essentially ensures that stack[:,:,1] (frame 1 in the stack of frames) corresponds roughly to stack[:,:,239] (frame 239 in the stack). Given this correspondence, a side-by-side combination of frames 1 and 239 should result in perfectly horizontal matches to be drawn across the images, seeing as these are nearly identical images that represent the same part of the brain.

Code taken from OpenCV's feature matching documentation (written in Python):

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread('box.png',0)          # queryImage
img2 = cv2.imread('box_in_scene.png',0) # trainImage

# Initiate ORB detector
orb = cv2.ORB_create()

# find the keypoints and descriptors with ORB
kp1, des1 = orb.detectAndCompute(img1,None)
kp2, des2 = orb.detectAndCompute(img2,None)

 # create BFMatcher object
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

# Match descriptors.
matches = bf.match(des1,des2)

# Sort them in the order of their distance.
matches = sorted(matches, key = lambda x:x.distance)

# Draw first 10 matches.
img3 = cv2.drawMatches(img1,kp1,img2,kp2,matches[:10], flags=2)

plt.imshow(img3),plt.show()
```

The code above illustrates an example of how to match images using the ORB algorithm (Oriented BRIEF) and a Brute-Force matcher to find matches between two images. This is done by setting some threshold for a match and calling those that passed the threshold "good"

matches. The brute-force matcher is one of multiple matches that can be used to perform match tests between two images.

Another method of performing the comparison is illustrated below. This is the FLANN (Fast Library for Approximate Nearest Neighbors) based Matcher. It is optimized for fast nearest-neighbor search in large datasets.[2]

```
import numpy as np
import cv2
from matplotlib import pyplot as plt

img1 = cv2.imread('box.png',0)          # queryImage
img2 = cv2.imread('box_in_scene.png',0) # trainImage

# Initiate SIFT detector
sift = cv2.SIFT()

# find the keypoints and descriptors with SIFT
kp1, des1 = sift.detectAndCompute(img1,None)
kp2, des2 = sift.detectAndCompute(img2,None)

# FLANN parameters
FLANN_INDEX_KDTREE = 0
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
search_params = dict(checks=50)   # or pass empty dictionary

flann = cv2.FlannBasedMatcher(index_params,search_params)

matches = flann.knnMatch(des1,des2,k=2)

# Need to draw only good matches, so create a mask
matchesMask = [[0,0] for i in xrange(len(matches))]

# ratio test as per Lowe's paper
for i,(m,n) in enumerate(matches):
    if m.distance < 0.7*n.distance:
        matchesMask[i]=[1,0]

draw_params = dict(matchColor = (0,255,0),
                   singlePointColor = (255,0,0),
                   matchesMask = matchesMask,
                   flags = 0)

img3 = cv2.drawMatchesKnn(img1,kp1,img2,kp2,matches,None,**draw_params)

plt.imshow(img3,),plt.show()
```

These two methods of performing matches will be crucial to deciding the ultimate, winning feature detection method.

2 Hypothesis

Based on prior work in the biological field, it is expected that feature-based registration will be ineffective for processing purposes.

Like other brain images, our brain images do not necessarily contain distinct features for the methods to be effective. Therefore, there is a possibility that these methods do not reveal any promising results. This, however, can only be confirmed by further exploration of even newer algorithms, which our research aims to do.

3 Methods

There were several steps involved in testing out feature-based registration. The first was visualizing the matches between two corresponding images in a stack of brain images. The two main tools used for feature-based registration were the Julia Programming Language and OpenCV (Open Source Computer Vision) software. Julia is a developing programming language (2012) specialized for scientific computing. Its ease of use and speed provided a useful means for processing and writing scripts. OpenCV provided the feature-detection algorithms and documentation for matching scripts. Based on these, I was able to test the effectiveness of several feature-detection methods: AGAST, AKAZE, BRISK, FAST, FREAK, SURF, SIFT, KAZE, and ORB.

The first step was visualizing the matches between two corresponding frames. In my case, those two corresponding frames were slice 59 and slice 182, based on the organization of the stack. For each of the methods listed, I tried to see if the perfectly horizontal matches could be found straight across the images. Whatever the case, I then changed several parameters and data structures in such a way that yielded perfect horizontal matches. I did this to see the potential of individual methods in yielding effective matches. Those methods that took too long to yield matches were eliminated immediately, for their lack of performance.

After I saw that a few had potential to yield effective matches, I expanded on their abilities. I tested whether these methods were able to find the correct corresponding matches themselves. In other words, if I fed the algorithm frame 59, could it find the corresponding frame - that is, 182? The number of matches is what determines whether the correct corresponding frame was found. The input image (called the query image) should contain the most number of matches against its corresponding frame. For example, if I tested input image slice 59 against the whole half-stack (slices 120 to 239), the most number of matches should be found between slice 59 and 182. That being said, to determine what corresponding frame was found, I recorded the frame for which the number of matches found was the greatest. I tested this out for both the total matches and “valid” matches. The concept of a valid match comes from the application of a sum of square error. We set our sum of square error (SSE) to 50 pixels, as this is roughly the size of a single cell in a brain image. This means that a match has a le-way of 50 pixels, and if it goes above this threshold, it is not considered a valid match. So, this threshold was set in order to identify which matches likely gave that perfectly horizontal line; valid matches are - in other words - accurate matches.
I then tested how effective the methods were in doing this for the total and valid matches. I studied the performance, as well as the number of matches found by different methods. Those that had a good performance and a good number of matches qualified to be further explored. After those were identified, I improved on them with different matchers provided by OpenCV

and found the best" version" amongst these improvements. To find the best" version", I looked for a reasonable trade-off between two things: accuracy and performance.
Those versions which had a high valid match to total match ratio, as well as good performance, were ideal.

After that, it was time to apply the test in 3-Dimensions. This meant using three coordinates: x, y, and z in the sum of square error. Now, when setting the input image to one slice, and finding matches against the half stack (frames 120 to 239), the z-slice those matches came from will be factored into the sum of square error. In the end, my goal was to determine what z-slice the input frame came from. In essence, this isn't at all different from finding the best-matching frame (the exact same result is expected); only this time, the Z was involved in the sum of square error test.

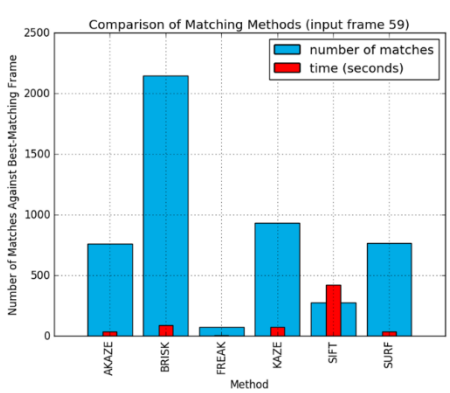
Once we determined that the correct z-slice could be found, we moved to applying a transform to the input image and repeating the test. First, this transform was a rotation. If the correct z-slice could be identified given a transformed image, we applied an Affine Transform to the input image, thus working with 6 degrees of freedom and tackling a very challenging problem in image registration.

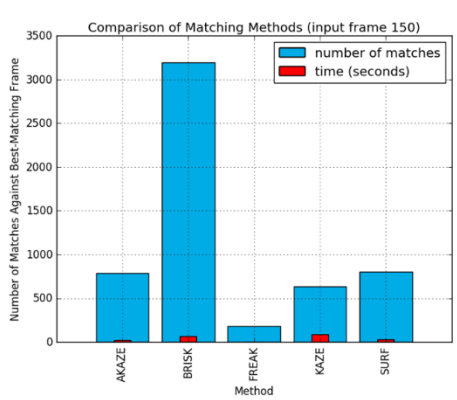
4 Results

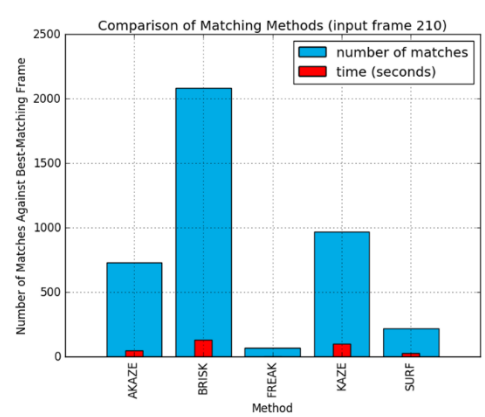
In the first step of changing parameters around to identify which methods could be potentially promising, we found that AGAST and FAST were very slow in finding matches. These were eliminated based on the poor performance.

In the second step of testing whether the best-matching frame could be found, we identified that all methods but Oriented BRIEF (ORB) could find the correct corresponding frame. ORB was eliminated, while the other methods were in the running.

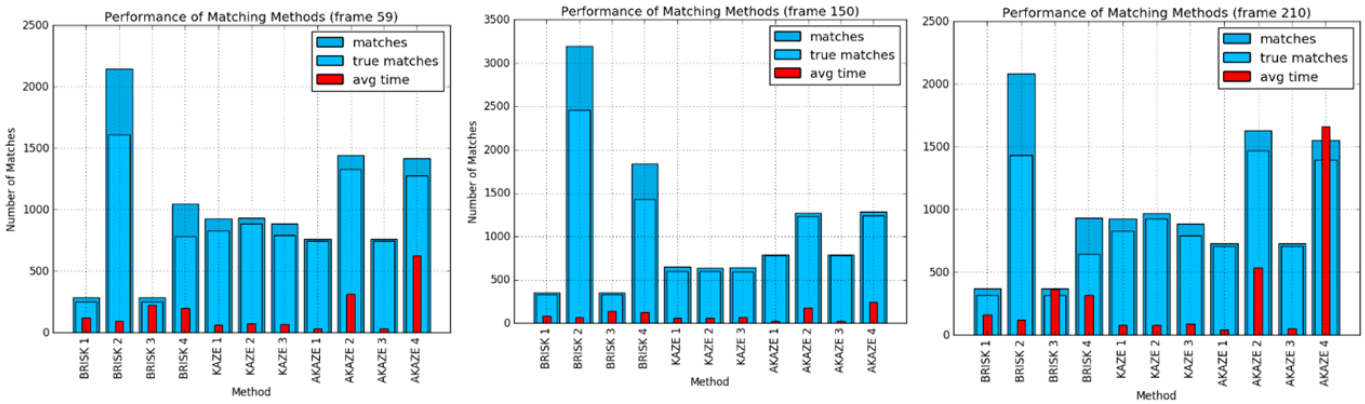
For all of the rest, we compared total number of matches and time for three input frames: 59, 150, and 210. In the winning method, we looked for methods that were consistently good in the number of matches they yielded, as well as performance. SIFT was too slow. FREAK yielded too few matches, and SURF was inconsistent across the tests. That left AKAZE, BRISK, and KAZE as the ones with which to expand.



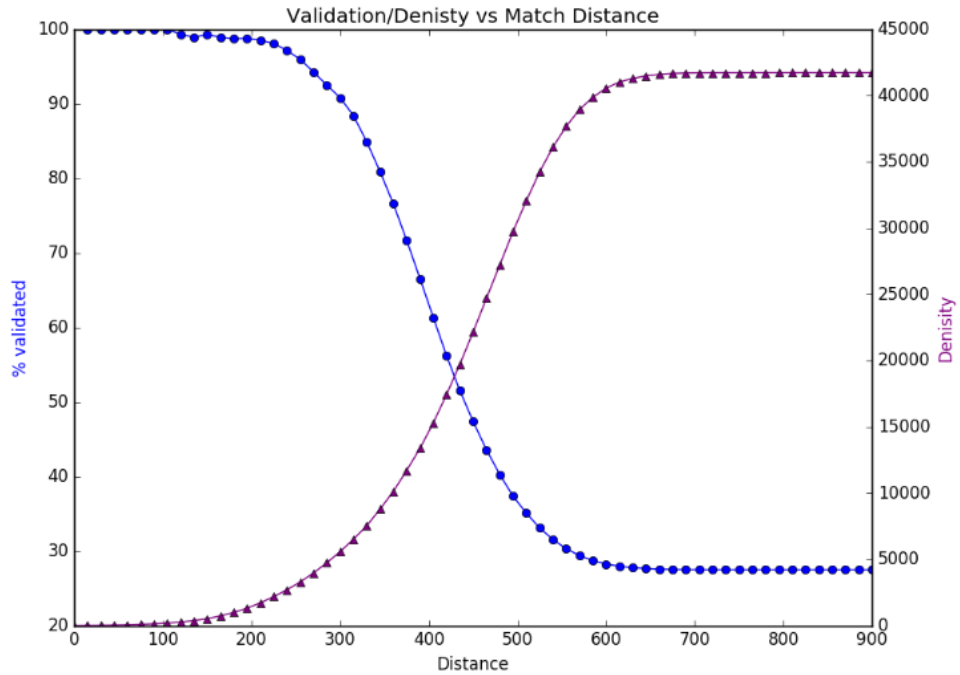




Consequently, we created different versions of these algorithms. One version was using the default implementation - no parameters changed and the ratio test applied. Another version was with the FLANN-based matcher with Kd-Trees applied. BRISK had a low accuracy rate. KAZE wasn't as good in performance as AKAZE. AKAZE-2 (the FLANN-based implementation with Kd-Trees) was the winning method.

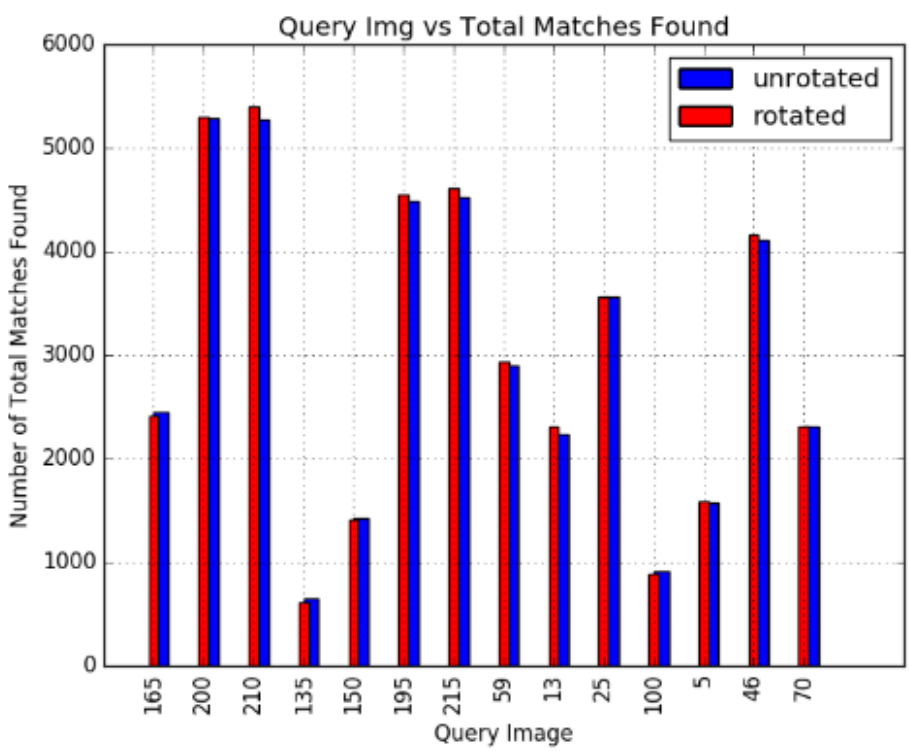


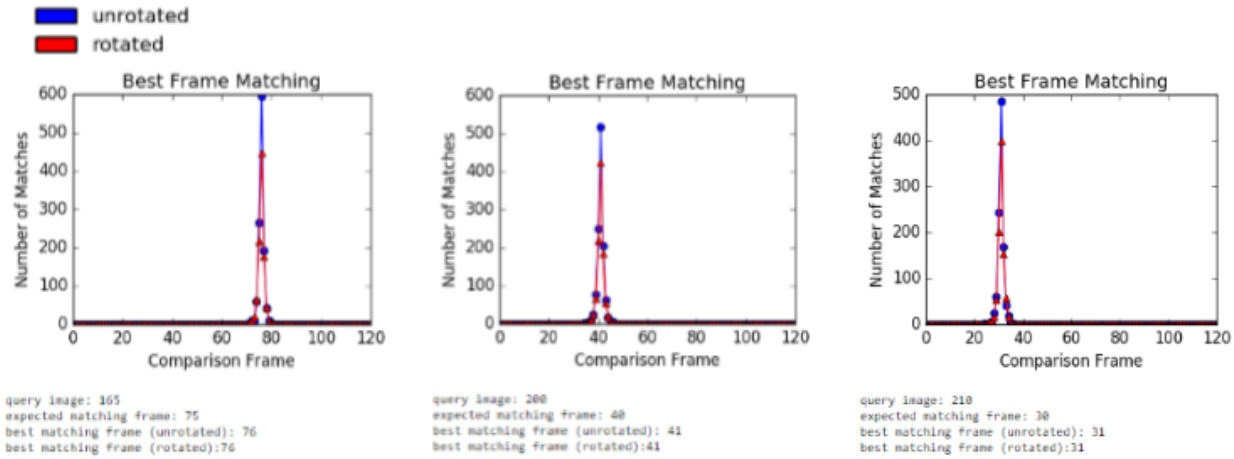
We then analyzed match distance and the number of validated matches. Based on the plot below, we can see that we need a match distance of about 450 to get a 50% chance of our matches being valid matches. A distance of 300 is where the percentage of matches validated starts to fall away from 100%.



When we applied the AKAZE-2 method to rotated and un-rotated images in the 3-D plane and got accurate results.

This bar graph shows the number of total matches found for both the rotated and un-rotated query image. As the data suggests, the number of matches remains relatively constant.

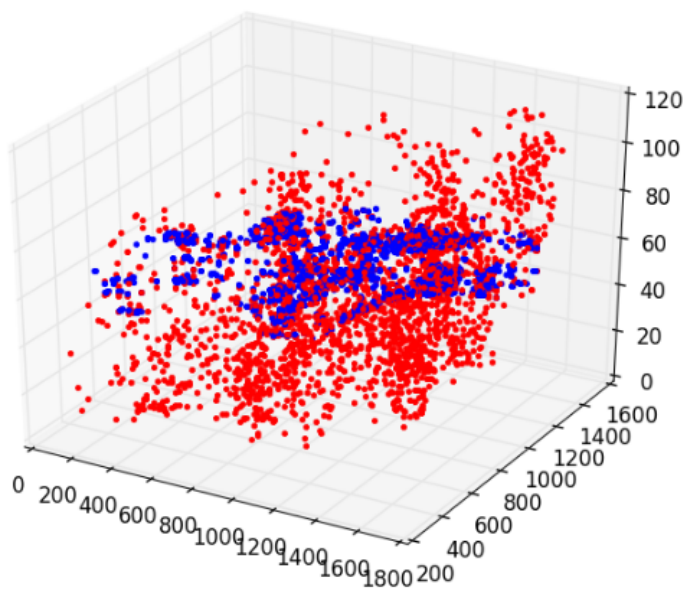




All in all, we see that these feature detection algorithms were able to find the corresponding best-matching frame, whether they were rotated or not.

The next step was to apply a transform and try to map the matches in a 3D Plot.

The following is what the matches look like with the transform function applied, and pitch = 0, and an input index of 59. What results is a significant number of matches (996 total), suggesting that matching methods are working with transformed images, as well.



5 Discussion

The AKAZE method looks to be the most efficient, robust method for our purposes. Other methods were excluded either for poor performance or for not matching up to the number of matches and efficiency of other methods. After using AKAZE to find matches against frames in the half stack with 3D-version and rotated input images, we think feature-based methods could be effective in the application with transformed images, as well.

The essential importance to note is that these results can be replicated quickly and through a mere few changes in parameters. It is possible that there are other, more efficient methods, seeing as the combination of parameters and data structures are infinite, but from our data, the accuracy and speed of AKAZE (Accelerated KAZE) seems to meet the demands of our registration requirements.

Because AKAZE is a relatively new method (published in 2013), it is a relatively new venture to compare it to other, popular feature detection algorithms. This means that this exploration and application is worthwhile, and could perhaps be expanded by the development by even more robust, efficient algorithms.

6 Conclusion

As our research suggests, feature-based registration does indeed have the potential of being effective in brain imaging. What was previously thought as futile, now shows promising results. The next step would be to try to estimate transforms from matches that are found and apply it in such a way that enables us to perform image alignment more efficiently. All in all, this proves to be a feasible application for registration of brain images.

7 Acknowledgements

This research was supported by the NSF (National Science Foundation) and through the Rowland Institute for Science at Harvard University.

8 References

[1] Teverovskiy, Leonid A., et al. *"Feature-based vs. intensity-based brain image registration: voxel level and structure level performance evaluation."* School of Computer Science Carnegie Mellon University 6 (2006): 1-30.

[2] Feature Matching, from *docs.opencv.org* 3.0-beta documentation/OpenCV Python Tutorials/Feature Detection and Description (2014).

[3] Alcantarilla, Pablo F., and TrueVision Solutions. "*Fast explicit diffusion for accelerated features in nonlinear scale spaces."* IEEE Trans. Patt. Anal. Mach. Intell 34.7 (2011): 1281-1298.

[4] Alcantarilla, Pablo Fernández, Adrien Bartoli, and Andrew J. Davison. "*KAZE features*." European Conference on Computer Vision. Springer Berlin Heidelberg, 2012.

[5] Leutenegger, Stefan, Margarita Chli, and Roland Y. Siegwart. "*BRISK: Binary robust invariant scalable keypoints.*" 2011 International conference on computer vision. IEEE, 2011.

[6] Sanderson, Conrad, and Kuldip K. Paliwal. "*Fast features for face authentication under illumination direction changes.*" Pattern Recognition Letters 24.14 (2003): 2409-2419

[7] Kneip, Laurent, Margarita Chli, and Roland Siegwart. "Robust real-time visual odometry with a single camera and an imu." BMVC. 2011.

[8] Li, Huifen, Xaingqian Jiang, and Zhu Li. "*Robust estimation in Gaussian filtering for engineering surface characterization*." Precision Engineering 28.2 (2004): 186-193.

[9]Rublee, Ethan, et al. "*ORB: An efficient alternative to SIFT or SURF*." 2011 International conference on computer vision. IEEE, 2011.

[10] Lowe, David G. "*Distinctive image features from scale-invariant keypoints*." International journal of computer vision 60.2 (2004): 91-110.

[11] Bay, Herbert, Tinne Tuytelaars, and Luc Van Gool. "*Surf: Speeded up robust features*." European conference on computer vision. Springer Berlin Heidelberg, 2006